

Resumo

Este handout apresenta os fundamentos da linguagem HTML (*HyperText Markup Language*), a linguagem de marcação que estrutura o conteúdo de páginas web. São abordados a anatomia das tags e seus atributos, a estrutura obrigatória de um documento HTML válido, o que compõe o cabeçalho (<head>) e o corpo (<body>), os principais elementos de texto, links, imagens e formulários, e os princípios do HTML semântico, que tornam páginas mais acessíveis, indexáveis e manuteníveis. A parte final conecta o HTML ao ecossistema Node.js/Express, mostrando como servir páginas estáticas, proteger rotas e gerar HTML dinâmico com template engines. CSS e JavaScript são mencionados brevemente como as camadas complementares de apresentação e comportamento, aprofundados em handouts próprios.

Como usar este material: ao longo do handout você encontrará boxes coloridos com diferentes funções. Boxes verdes trazem dicas práticas de desenvolvimento. Boxes amarelos propõem exercícios de fixação. Boxes vermelhos alertam para armadilhas comuns. Boxes azuis apresentam informações contextuais importantes. Todo código deve ser digitado — não copiado — para consolidar o aprendizado.

1. O Que É HTML

HTML (*HyperText Markup Language*) é a linguagem de marcação padrão da web. Ela não é uma linguagem de programação — não tem variáveis, laços ou condicionais. Seu papel é descrever a **estrutura** e o **significado** do conteúdo: dizer ao navegador o que é um parágrafo, o que é um título, o que é uma imagem e onde fica o rodapé.

O termo *marcação* (markup) vem da tradição editorial: revisores usavam marcas físicas nos manuscritos para indicar ao tipógrafo como formatar o texto. O HTML faz o mesmo de forma digital e padronizada: envolve o conteúdo em **tags** que descrevem seu papel semântico. O navegador lê essas marcações, constrói uma representação interna do documento — o DOM (*Document Object Model*) — e a exibe ao usuário.

Uma breve história do HTML Tim Berners-Lee criou o HTML em 1991 no CERN, como uma forma simples de compartilhar documentos científicos pela internet. A versão 4.01, de 1999, consolidou a linguagem; o HTML5, publicado em 2014 pelo W3C em conjunto com o WHATWG, é a versão atual e introduziu elementos semânticos, suporte nativo a vídeo e áudio, APIs JavaScript avançadas e muito mais. O HTML5 é uma especificação viva: continua evoluindo sem versões numeradas fixas.

Uma página web é composta por três camadas complementares, cada uma com responsabilidade distinta:

Camada	Tecnologia	Responsabilidade
Estrutura	HTML	Define o que o conteúdo <i>é</i> : títulos, parágrafos, listas, imagens.
Apresentação	CSS	Define como o conteúdo <i>aparece</i> : cores, fontes, layout.
Comportamento	JavaScript	Define como o conteúdo <i>age</i> : interações, chamadas a APIs.

Tabela 1. As três camadas de uma página web.

Separação de responsabilidades na web Assim como o princípio SoC (*Separation of Concerns*) orienta a arquitetura de *back-end*, ele também se aplica ao *front-end*. Misturar estilos CSS diretamente nas tags HTML (atributo `style="..."`) ou embutir *scripts* no meio do conteúdo dificulta a manutenção e impede o reuso. O ideal é manter cada camada em seus próprios arquivos: `.html`, `.css` e `.js`.

2. Anatomia de uma Tag HTML

O bloco fundamental do HTML é a **tag**. A maioria das tags tem uma versão de abertura e uma de fechamento, envolvendo um conteúdo entre elas. Esse conjunto — tag de abertura, conteúdo e tag de fechamento — é chamado de **elemento**. O navegador analisa os elementos em sequência e os organiza em uma árvore hierárquica, o DOM.

```
<!-- Tag de abertura + conteúdo + tag de fechamento = elemento -->
<p>Este é um parágrafo.</p>

<!-- Tags de fechamento têm uma / antes do nome -->
<h1>Título principal</h1>
<strong>texto em negrito semântico</strong>
<a href="https://uffs.edu.br">Link para a UFFS</a>
```

Código 1. Anatomia de um elemento HTML.

Nomes de tags são escritos em letras minúsculas por convenção. O navegador é tolerante com maiúsculas — `<P>` e `<p>` produzem o mesmo resultado — mas a convenção minúscula é universal e deve sempre ser seguida.

2.1. Atributos

Os atributos fornecem informações adicionais sobre um elemento. Eles são escritos **dentro da tag de abertura**, no formato `nome="valor"`. Um elemento pode ter zero, um ou vários atributos; a ordem deles não importa para o navegador, mas uma ordem consistente melhora a legibilidade.

```
<!-- Um atributo: href define o destino do link -->
<a href="https://uffs.edu.br">Site da UFFS</a>

<!-- Múltiplos atributos na mesma tag -->
<a href="https://uffs.edu.br" target="_blank" rel="noopener">
  Abrir em nova aba
</a>

<!-- Atributos booleanos: a presença já ativa o comportamento -->
<input type="checkbox" checked>
<input type="text" disabled>
```

Código 2. Atributos: sintaxe e exemplos básicos.

Atributos booleanos — como `checked`, `disabled`, `required` e `hidden` — não precisam de valor: a simples presença da palavra já ativa o comportamento. Escrever `disabled="disabled"` ou `disabled="true"` é válido, mas desnecessário.

Atributo	Função
<code>id</code>	Identificador único no documento. Usado por CSS (<code>#id</code>) e JavaScript.
<code>class</code>	Uma ou mais classes CSS para estilizar grupos de elementos.
<code>style</code>	CSS inline. Útil para testes rápidos, mas evitar em produção.
<code>lang</code>	Idioma do conteúdo (<code>lang="pt-BR"</code>). Importante para leitores de tela.
<code>title</code>	Texto exibido como <i>tooltip</i> ao passar o mouse.
<code>hidden</code>	Oculto o elemento (equivalente a <code>display: none</code>).
<code>tabindex</code>	Controla a ordem de foco ao navegar com a tecla Tab.
<code>data-*</code>	Atributos personalizados para armazenar dados extras sem semântica HTML.

Tabela 2. Atributos globais — válidos em qualquer elemento HTML.

Os atributos `id` e `class` são os mais frequentemente usados no desenvolvimento web. `id` deve ser único na página — dois elementos não podem compartilhar o mesmo `id`. `class` pode se repetir: múltiplos elementos podem ter a mesma classe, e um único elemento pode ter múltiplas classes separadas por espaço: `class="card destaque ativo"`.

Os atributos `data-*` permitem armazenar dados arbitrários nos elementos sem violar a especificação HTML. Por exemplo, `data-id="42"` pode ser lido em JavaScript com `elemento.dataset.id`. Esse padrão é amplamente usado para passar informações do HTML para o JavaScript sem poluir o código com variáveis globais.

2.2. Tags sem conteúdo (void elements)

Algumas tags não envolvem conteúdo e, portanto, não têm tag de fechamento. Esses elementos são chamados de *void elements* (ou elementos vazios) e realizam funções que não envolvem texto — inserir uma imagem, uma quebra de linha ou um campo de formulário, por exemplo.

```

<input type="text" name="nome" placeholder="Seu nome">
<br>    <!-- quebra de linha -->
<hr>    <!-- linha horizontal separadora -->
<meta charset="UTF-8">
<link rel="stylesheet" href="estilo.css">
```

Código 3. Elementos HTML sem tag de fechamento.

⚠ Não feche void elements com `/>`. Em HTML5, escrever `<br />` ou `<img ... />` é válido mas desnecessário — herança do XHTML. A forma recomendada em HTML5 é simplesmente `<br>` e `<img>`.

2.3. Aninhamento e indentação

Elementos HTML podem ser aninhados uns dentro dos outros, formando uma estrutura de árvore chamada **DOM** (*Document Object Model*). A regra fundamental é: **elementos devem ser fechados na ordem inversa em que foram abertos**. O descumprimento dessa regra cria HTML malformado que os navegadores tentam corrigir de maneiras imprevisíveis — o que pode resultar em layouts quebrados difíceis de depurar.

```
<!-- Correto: fechamento na ordem inversa de abertura -->
<p>Texto com <strong>negrito <em>e itálico</em></strong> juntos.</p>

<!-- Incorreto: sobreposição de tags -->
<p>Texto com <strong>negrito <em>e itálico</strong></em> juntos.</p>

<!-- Lista com itens devidamente aninhados -->
<ul>
  <li>Item um</li>
  <li>Item dois
    <ul><li>Subitem</li></ul>
  </li>
</ul>
```

Código 4. Aninhamento correto e incorreto de elementos.

💡 Indentação revela a estrutura. Use dois ou quatro espaços de indentação para cada nível de aninhamento. Além de melhorar a legibilidade, a indentação torna erros de fechamento de tags imediatamente visíveis. Editores como o VS Code oferecem formatação automática de HTML com `Shift+Alt+F` (Windows/Linux) ou `Shift+Option+F` (macOS).

3. Estrutura de um Documento HTML

Todo documento HTML válido segue uma estrutura obrigatória composta por quatro partes: a declaração de tipo de documento, o elemento raiz `<html>`, o cabeçalho `<head>`

e o corpo `<body>`. O navegador depende dessa estrutura para interpretar e renderizar a página corretamente. Omitir qualquer parte não impede o navegador de exibir a página — navegadores são notoriamente tolerantes — mas pode causar comportamentos inesperados em diferentes ambientes e dispositivos.

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Descrição para mecanismos de busca.">
    <title>Título da Página exibido na aba</title>
    <link rel="stylesheet" href="estilo.css">
  </head>

  <body>
    <header>
      <nav>...</nav>
    </header>

    <main>
      <h1>Título principal</h1>
      <p>Conteúdo da página.</p>
    </main>

    <footer>...</footer>

    <!-- Scripts no final do body: não bloqueiam o carregamento -->
    <script src="script.js"></script>
  </body>
</html>
```

Código 5. Estrutura do documento HTML5 — cabeçalho.

Código 6. Estrutura do documento HTML5 — corpo.

3.1. O DOCTYPE e o elemento html

A declaração `<!DOCTYPE html>` na primeira linha informa ao navegador que o documento usa HTML5. Não é uma tag HTML — não tem tag de fechamento e não aparece no DOM. Sem ela, o navegador pode entrar em “modo quirks” (*quirks mode*), emulando comportamentos de navegadores dos anos 1990 para manter compatibilidade com sites antigos. Isso significa layouts quebrados, comportamentos de CSS inesperados e resultados diferentes entre navegadores.

O elemento `<html>` é a raiz do documento e deve envolver todo o conteúdo. O atributo `lang` é obrigatório por boas práticas: informa o idioma principal do conteúdo, usado por leitores de tela para selecionar a pronúncia correta, por ferramentas de tradução automática e por mecanismos de busca para segmentação geográfica. Use `lang="pt-BR"` para português do Brasil, `lang="en"` para inglês.

Para pensar: Qual é a diferença entre o modo quirks e o modo padrão do navegador? O que acontece exatamente quando o `<!DOCTYPE html>` é omitido? Pesquise o termo “browser rendering modes” e identifique ao menos dois comportamentos de CSS que diferem entre os dois modos.

4. O Cabeçalho: <head>

O elemento `<head>` contém **metadados** — informações sobre o documento que não são exibidas diretamente na página, mas são essenciais para mecanismos de busca, redes sociais, navegadores e leitores de tela. O que vai no `<head>` afeta diretamente como a página é encontrada, compartilhada e renderizada — aspectos invisíveis ao olho nu, mas críticos para o sucesso de qualquer aplicação web.

```
<head>
  <!-- 1. Codificação: sempre o primeiro meta -->
  <meta charset="UTF-8">
  <!-- 2. Viewport: essencial para responsividade móvel -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Código 7. Metadados essenciais no head.

```
<!-- 3. Título: exibido na aba e nos resultados de busca (max. 60 chars) -->
<title>GEX613 Programação II | UFFS</title>
<!-- 4. Descrição: resumo para mecanismos de busca (50160 chars) -->
<meta name="description"
      content="Disciplina de Programação II UFFS Chapecó.">
</head>
```

O `<meta charset="UTF-8">` deve ser o **primeiro** elemento dentro do `<head>`, antes do `<title>`. Isso garante que o navegador conheça a codificação de caracteres antes de tentar exibir qualquer texto — incluindo o próprio título da página. UTF-8 é a codificação padrão da web e suporta todos os caracteres do Unicode: acentos, caracteres especiais, emojis e alfabetos de qualquer língua.

O `<meta name="viewport">` controla como a página é exibida em dispositivos móveis. Sem ele, navegadores de celular renderizam a página como se tivesse a largura de um monitor de desktop e depois a encolhem — o que resulta em texto minúsculo ilegível. `width=device-width` faz a página ocupar a largura real do dispositivo; `initial-scale=1.0` define o zoom inicial como 100%.

```
<head>
  <!-- Folha de estilo CSS -->
  <link rel="stylesheet" href="estilo.css">
  <!-- Ícone da aba (favicon) -->
  <link rel="icon" type="image/png" href="favicon.png">
  <!-- Fonte externa (Google Fonts) -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=Inter&display=swap"
        rel="stylesheet">
</head>
```

Código 8. Recursos externos: CSS, favicon e fontes.

Colocar no head	Evitar no head
<code><meta charset></code> e <code><meta viewport></code>	Conteúdo visível ao usuário
<code><title></code> único e descritivo por página	<code><script></code> que manipulam o DOM
<code><meta name="description"></code>	Estilos <code><style></code> extensos
<code><link rel="stylesheet"></code>	Lógica de negócio ou scripts pesados
<code><link rel="icon"></code> (favicon)	

Tabela 3. O que colocar e o que evitar no `<head>`.

💡 O título da página é o elemento mais importante para SEO. O conteúdo da tag `<title>` aparece como título clicável nos resultados do Google e como nome da aba no navegador. Cada página deve ter um título único e descritivo, com no máximo 60 caracteres. Um bom padrão é Nome da Página – Nome do Site. Títulos genéricos como “Página inicial” ou “Untitled” prejudicam tanto a usabilidade quanto o posicionamento em buscas.

5. O Corpo: `<body>`

O elemento `<body>` contém todo o conteúdo visível da página — textos, imagens, links, vídeos, formulários. A organização interna deve seguir uma hierarquia lógica e semântica: não apenas uma sequência de elementos, mas uma estrutura que comunique o significado e a relação entre as partes do conteúdo.

5.1. Elementos de texto

Os elementos de texto são os mais fundamentais do HTML. A distinção entre elementos puramente visuais e elementos com significado semântico é central: use `<strong>` em vez de `<b>` quando o texto for importante, e `<em>` em vez de `<i>` quando for ênfase — a diferença não é visual, é semântica, e afeta leitores de tela e mecanismos de busca.

```
<!-- h1 é o mais importante, h6 o menos. Um único h1 por página. -->
<h1>Título principal da página</h1>
<h2>Seção principal</h2>
```

Código 9. Títulos, parágrafos e ênfase semântica.

```
<h3>Subseção</h3>

<p>0 parágrafo é o elemento de texto mais comum.</p>

<!-- strong e em têm significado semântico -->
<p>Use <strong>strong</strong> para importância.</p>
<p>Use <em>em</em> para ênfase ou entonação.</p>
<!-- <b> e <i> são puramente visuais prefira strong/em -->
```

Além de `<strong>` e `<em>`, o HTML oferece outros elementos inline com semântica específica: `<code>` para trechos de código, `<kbd>` para teclas do teclado, `<mark>` para texto destacado por relevância, `<del>` para texto excluído e `<ins>` para texto inserido. Cada um comunica uma intenção diferente que um `<span>` com CSS não é capaz de expressar.

```
<ul>                                <!-- lista sem ordem específica -->
  <li>Item um</li>
  <li>Outro item</li>
</ul>

<ol>                                <!-- lista com ordem importa -->
  <li>Primeiro passo</li>
  <li>Segundo passo</li>
</ol>

<dl>                                <!-- termos e descrições -->
  <dt>HTML</dt>
  <dd>Linguagem de marcação para estruturar páginas web.</dd>
  <dt>CSS</dt>
  <dd>Linguagem de estilos para apresentação visual.</dd>
</dl>
```

Código 10. Listas não ordenadas, ordenadas e de definição.

A escolha entre `<ul>` e `<ol>` não é estética — é semântica. Use `<ol>` quando a ordem dos itens importa (passos de instalação, ranking, receita culinária). Use `<ul>` quando a ordem é irrelevante (lista de funcionalidades, ingredientes, links de navegação). Listas podem ser aninhadas: um `<li>` pode conter outro `<ul>` ou `<ol>` dentro. A lista de definição `<dl>` é subutilizada, mas ideal para glossários, metadados de artigos e pares chave-valor.

5.2. Links

O elemento âncora `<a>` é o que torna a web uma *teia* de documentos interconectados. O atributo `href` (*hypertext reference*) define o destino do link e aceita URLs absolutas, relativas, âncoras internas à página e esquemas especiais como `mailto:` e `tel:`.

```
<!-- Link externo em nova aba: rel="noopener" é obrigatório por segurança -->
<a href="https://uffs.edu.br" target="_blank" rel="noopener noreferrer">
  UFFS (nova aba)
</a>

<!-- Âncora para seção da mesma página -->
<a href="#contato">Ir para Contato</a>
<section id="contato">...</section>

<!-- Link para e-mail, telefone e download -->
<a href="mailto:gian@uffs.edu.br">gian@uffs.edu.br</a>
<a href="tel:+554936218000">Ligar para a UFFS</a>
<a href="programa.pdf" download>Baixar programa</a>
```

Código 11. O elemento anchor e seus atributos principais.

⚠ `target="_blank"` sem `rel="noopener"` é uma vulnerabilidade. Quando um link abre em nova aba com `target="_blank"`, a página de destino pode acessar o objeto `window.opener` da página de origem e redirecioná-la para outro endereço — um ataque chamado *tabnapping*. O atributo `rel="noopener noreferrer"` impede esse acesso. Sempre use os dois juntos ao abrir links em nova aba.

⚠ Nunca use "clique aqui" como texto de link. Textos como “clique aqui” ou “saiba mais” são inacessíveis para usuários de leitores de tela e prejudicam o SEO. Um leitor de tela pode listar todos os links da página; se todos forem “clique aqui”, o usuário não sabe para onde ir. Use sempre textos descritivos: `<a href="...">Programa da disciplina GEX613</a>`.

5.3. Imagens

O elemento `<img>` é um *void element*: não tem tag de fechamento e carrega a imagem definida pelo atributo `src`. Os atributos `src` e `alt` são obrigatórios — o primeiro define o caminho da imagem, o segundo fornece uma descrição textual alternativa.

```
<!-- src: caminho. alt: texto alternativo (OBRIGATÓRIO). -->


<!-- Imagem decorativa: alt vazio leitores de tela ignoram -->


<!-- Imagem com link: o alt deve descrever o destino -->
<a href="https://uffs.edu.br">
  
</a>
```

Código 12. O elemento `img`: atributos obrigatórios e responsividade.

Os atributos `width` e `height` devem refletir as dimensões reais da imagem em pixels. O motivo é o *Cumulative Layout Shift* (CLS): ao conhecer as dimensões antes de baixar a imagem, o navegador reserva o espaço correto na página, evitando que o conteúdo “pule” durante o carregamento — o que prejudica a experiência do usuário e as métricas de performance do Google (Core Web Vitals).

💡 O atributo `alt` deve ser significativo. O `alt` serve a três propósitos: (1) leitores de tela o leem para usuários com deficiência visual; (2) é exibido quando a imagem não carrega; (3) mecanismos de busca o usam para indexar imagens. Descreva o conteúdo e o contexto: `alt="Foto"` é inútil; `alt="Alunos em aula prática no laboratório da UFFS"` é correto. Para imagens puramente decorativas, use `alt=""` — isso instrui o leitor de tela a ignorar a imagem completamente.

5.4. Formulários

Formulários são o mecanismo principal de coleta de dados do usuário. O elemento `<form>` define o contêiner do formulário; dentro dele ficam os campos (`<input>`, `<textarea>`, `<select>`) e os botões. O atributo `action` define para onde os dados serão enviados; `method` define o verbo HTTP (GET ou POST).

```
<form action="/login" method="POST">
  <!-- label associado ao input via for = id -->
  <label for="email">E-mail:</label>
  <input type="email" id="email" name="email"
        placeholder="seu@email.com" required
        autocomplete="email">

  <label for="senha">Senha:</label>
  <input type="password" id="senha" name="senha"
        required minlength="8"
        autocomplete="current-password">

  <button type="submit">Entrar</button>
</form>
```

Código 13. Estrutura básica de formulário acessível.

O atributo `name` define a chave com que o dado será enviado ao servidor. Em uma requisição POST para `/login`, o servidor receberá `email=valor` e `senha=valor`. O

atributo `required` ativa a validação nativa do navegador antes de submeter o formulário. O atributo `autocomplete` sugere ao navegador qual informação preencher automaticamente — um detalhe simples que melhora significativamente a experiência do usuário.

O HTML5 introduziu vários tipos de `<input>` com validação e interface adaptada ao dispositivo: `type="email"` valida o formato de e-mail; `type="number"` exibe teclado numérico em celulares; `type="date"` exibe um seletor de data nativo. Use sempre o tipo mais específico disponível — isso melhora a usabilidade sem escrever uma linha de JavaScript.

⚠ Sempre associe labels aos inputs. Um `<input>` sem `<label>` associado é inacessível. A associação é feita pelo par `for="id-do-input"` no label e `id="mesmo-id"` no input. Usar placeholder como substituto do label é um erro comum: o placeholder desaparece ao digitar e não é anunciado por todos os leitores de tela. Mantenha sempre o `<label>` visível — mesmo que a estética peça um campo sem legenda, resolva isso com CSS, não removendo o label.

6. HTML Semântico

HTML semântico significa usar a tag que melhor descreve o **significado** do conteúdo, não apenas a aparência desejada. Uma `<div>` e um `<article>` podem ter a mesma aparência visual com CSS, mas o `<article>` comunica que o conteúdo é um bloco independente e autocontido — informação que navegadores, leitores de tela e mecanismos de busca entendem e utilizam.

Antes do HTML5, toda a estrutura de páginas era feita com `<div>`: `<div id="header">`, `<div id="nav">`, `<div id="footer">`. Esse padrão — conhecido como *div soup* — é tecnicamente funcional, mas semanticamente vazio: um agente automatizado (leitor de tela, *crawler* de busca) não consegue distinguir o conteúdo principal do cabeçalho ou do rodapé. O HTML5 introduziu elementos que comunicam essa estrutura de forma nativa.

6.1. Estrutura semântica de página

O HTML5 introduziu elementos que substituem o uso indiscriminado de `<div>`. Os dois blocos a seguir mostram a estrutura externa e o conteúdo principal de uma página real:

```
<body>
  <header>                                <!-- cabeçalho do site -->
    
    <h1>Programação II GEX613</h1>
  </header>

  <nav aria-label="Navegação principal">
    <ul>
      <li><a href="/home">Início</a></li>
      <li><a href="/aulas">Aulas</a></li>
      <li><a href="/avaliações">Avaliações</a></li>
    </ul>
  </nav>

  <main>...</main>                        <!-- conteúdo principal: único por página -->

  <footer>                                <!-- rodapé do site -->
    <p>UFFS Campus Chapecó
      <a href="mailto:gian@uffs.edu.br">gian@uffs.edu.br</a></p>
  </footer>
</body>
```

Código 14. Estrutura semântica: header, nav, main, footer.

O atributo `aria-label` no elemento `<nav>` é importante quando há mais de uma navegação na página (principal e de rodapé, por exemplo): ele distingue as duas para leitores de tela. ARIA (*Accessible Rich Internet Applications*) é um conjunto de atributos que complementam a semântica HTML para contextos onde a marcação nativa não é suficiente.

```

<main>
  <!-- section: agrupa conteúdo com tema próprio -->
  <section aria-labelledby="titulo-handouts">
    <h2 id="titulo-handouts">Handouts da Disciplina</h2>

    <!-- article: conteúdo autocontido e distribuível -->
    <article>
      <h3>Handout 01 Introdução ao Node.js</h3>
      <p>Publicado em
        <time datetime="2026-03-10">10 de março de 2026</time>.</p>
      <a href="handout01.pdf">Baixar handout</a>
    </article>
  </section>

```

Código 15. section e article: conteúdo agrupado e autocontido.

A distinção entre `<section>` e `<article>` é sutil mas importante. Um `<article>` deve fazer sentido de forma independente — pode ser extraído da página e publicado em outro lugar (um post de blog, uma notícia, um produto de e-commerce). Uma `<section>` é apenas uma divisão temática dentro de um contexto maior — não faz necessariamente sentido sozinha.

```

<!-- aside: conteúdo relacionado mas não central -->
<aside aria-label="Informações da disciplina">
  <h2>Informações</h2>
  <dl>
    <dt>Professor</dt><dd>Prof. Dr. Giancarlo D. Salton</dd>
    <dt>Créditos</dt><dd>4</dd>
    <dt>Horário</dt><dd>Terças e quintas, 10h12h</dd>
  </dl>
</aside>
</main>

```

Código 16. aside: conteúdo tangencial ao principal.

6.2. Guia dos elementos semânticos

Elemento	Quando usar
<code><header></code>	Cabeçalho do site ou de uma seção. Contém logo, título e navegação.
<code><nav></code>	Conjunto de links de navegação principal ou secundária.
<code><main></code>	Conteúdo principal e único da página. Deve haver apenas um por página.
<code><section></code>	Agrupamento temático com título próprio dentro da página.
<code><article></code>	Conteúdo autocontido: post de blog, notícia, cartão de produto.
<code><aside></code>	Conteúdo tangencial: barras laterais, notas, glossários.
<code><footer></code>	Rodapé do site ou de uma seção. Autoria, links secundários, copyright.
<code><figure></code>	Conteúdo autocontido referenciado no texto: imagem, diagrama, código.
<code><figcaption></code>	Legenda para um <code><figure></code> .
<code><time></code>	Data ou hora legível por máquina via atributo <code>datetime</code> .
<code><address></code>	Informações de contato do autor ou organização.
<code><mark></code>	Texto destacado por relevância no contexto atual.
<code><abbr></code>	Abreviações. Use <code>title</code> para a forma completa.

Tabela 4. Elementos semânticos do HTML5 e quando usar cada um.

O elemento `<figure>` com `<figcaption>` é particularmente subutilizado. Ele serve para qualquer conteúdo referenciado no texto — não apenas imagens, mas também trechos de código, gráficos e citações. O `<figcaption>` associa uma legenda ao conteúdo de forma semântica, algo que uma `<p>` logo abaixo de uma `<img>` não consegue comunicar.

6.3. Div e Span: elementos neutros

`<div>` e `<span>` são elementos **sem semântica** — servem apenas para agrupamento e aplicação de estilos quando não existe um elemento semântico adequado:

```
<!-- Anti-padrão: div soup tudo em divs sem semântica -->
<div class="cabecalho">
  <div class="navegacao">
    <div class="item"><a href="/">Início</a></div>
  </div>
</div>

<!-- Correto: elementos semânticos -->
<header>
  <nav>
    <ul><li><a href="/">Início</a></li></ul>
  </nav>
</header>

<!-- span: inline, sem semântica, para estilizar partes do texto -->
<p>Carga horária de <span class="destaque">60 horas</span> semestrais.</p>
```

Código 17. div e span: uso correto vs. anti-padrão.

Por que o HTML semântico importa HTML semântico beneficia três grupos ao mesmo tempo. **(1) Acessibilidade:** leitores de tela anunciam `<nav>` como “navegação” e `<main>` como “conteúdo principal”, permitindo navegação rápida por seções. **(2) SEO:** mecanismos de busca indexam `<article>` e `<main>` com peso maior do que `<div>`s genéricas. **(3) Manutenção:** código semântico é mais legível — encontrar `<header>` é mais rápido do que procurar `<div class="header">`.

6.4. Boas práticas com títulos

A hierarquia de títulos (`h1` a `h6`) é uma das estruturas mais importantes para acessibilidade e SEO. Usuários de leitores de tela navegam de título em título como uma tabela de conteúdos — é a forma mais eficiente de se orientar em uma página longa. A hierarquia deve ser linear e sem saltos: depois de um `<h2>`, o próximo nível deve ser `<h3>`, nunca `<h4>` ou `<h5>` diretamente.

```
<!-- Correto: hierarquia linear sem pular níveis -->
<h1>Ciência da Computação UFFS</h1>
<h2>Disciplinas do 4º Semestre</h2>
<h3>GEX613 Programação II</h3>
<h4>Unidade 1: Node.js e Express</h4>

<!-- Incorreto: pula níveis -->
<h1>Ciência da Computação</h1>
<h3>GEX613</h3> <!-- pulou h2 -->
<h5>Unidade 1</h5> <!-- pulou h4 -->

<!-- Incorreto: heading apenas para deixar texto grande -->
<h3>Aviso importante</h3> <!-- use CSS para tamanho, não headings -->
```

Código 18. Hierarquia correta e incorreta de títulos.

⚠ Um único h1 por página — que descreva o conteúdo. Cada página deve ter exatamente um `<h1>`, específico para o conteúdo daquela página. Não use `<h1>` para o nome do site em todas as páginas — o Google interpretará que todas tratam do mesmo assunto. O nome do site vai na `<title>` e no `alt` do logo. O `<h1>` deve ser único e descritivo por página.

Para pensar: Acesse qualquer site que você use frequentemente e abra o Dev-Tools (F12). No painel **Elements**, procure a estrutura HTML e identifique: (1) quantos `<h1>` existem na página? (2) a hierarquia de títulos é linear? (3) o site usa elementos semânticos (`<header>`, `<nav>`, `<main>`) ou apenas `<div>`s? Instale

a extensão “Headings Map” no Chrome para visualizar a estrutura de títulos de qualquer página de forma visual.

7. CSS e JavaScript: Uma Breve Introdução

HTML define *o que* o conteúdo é. As outras duas camadas definem *como* ele aparece e *como* ele se comporta. Compreender como incluir CSS e JavaScript em uma página HTML é parte fundamental do desenvolvimento web — mesmo que os detalhes de cada linguagem sejam estudados em handouts dedicados.

7.1. CSS — Cascading Style Sheets

CSS controla a apresentação visual: cores, fontes, tamanhos, espaçamentos e layout. A forma correta de incluir CSS é via arquivo externo referenciado no `<head>`. Isso separa a apresentação da estrutura e permite que uma única folha de estilo seja reutilizada por todas as páginas do site — se precisar mudar a fonte, você altera um único arquivo, não centenas de páginas.

```
<!-- Preferível: arquivo externo (reutilizável em múltiplas páginas) -->
<link rel="stylesheet" href="estilo.css">

<!-- Aceitável para estilos específicos da página -->
<style>
  body { font-family: Arial, sans-serif; }
</style>

<!-- Evitar: CSS inline (dificulta manutenção, impossível reutilizar) -->
<p style="color: red; font-size: 18px;">Texto vermelho</p>
```

Código 19. Formas de incluir CSS.

```
/* Seletor de elemento: aplica a todos os <body> */
body {
  font-family: 'Inter', Arial, sans-serif;
  max-width: 960px;
  margin: 0 auto; /* centraliza a página */
  color: #333;
}

/* Seletor de classe */
.destaque { background-color: #fff3cd; font-weight: bold; }

/* Seletor de ID */
#titulo-principal { border-bottom: 2px solid #2e75b6; }

/* Seletor aninhado: <a> dentro de <nav> */
nav a { text-decoration: none; color: #1f4e79; }
```

Código 20. Exemplo mínimo de CSS.

CSS é abordado em handout específico Seletores, especificidade, o modelo de caixa (*box model*), Flexbox, Grid e responsividade são tópicos do handout de CSS. Aqui o importante é entender que CSS e HTML são linguagens separadas — cada uma com sua sintaxe e seu arquivo — e que a separação entre estrutura e apresentação é uma prática fundamental.

7.2. JavaScript — Comportamento e Interatividade

JavaScript é a linguagem de programação da web. Enquanto HTML e CSS são declarativos — descrevem o que existe e como aparece —, JavaScript é imperativo: executa lógica, responde a eventos e se comunica com servidores. A forma correta de incluir JavaScript é via arquivo externo, preferencialmente com o atributo `defer`, que garante que o script seja executado apenas após o HTML ser completamente analisado.

```
<!-- Preferível: arquivo externo com defer no <head> -->
<script src="script.js" defer></script>
```

Código 21. Formas de incluir JavaScript.

```
<!-- Também correto: arquivo externo no final do body -->
<script src="script.js"></script>
```

```
<!-- Evitar: script embutido no meio do HTML -->
<button onclick="alert('clique!')">Clique</button>
```

```
// script.js seleciona elemento e responde a evento
const botao = document.getElementById('btn-tema');

botao.addEventListener('click', function () {
  // Alterna a classe 'tema-escuro' no <body>
  document.body.classList.toggle('tema-escuro');
});
```

Código 22. Exemplo mínimo: JavaScript manipulando o DOM.

JavaScript é abordado em handout específico Variáveis, funções, eventos, manipulação do DOM e integração com APIs REST são tópicos do handout de JavaScript. O importante aqui é entender que JavaScript acessa e modifica o HTML via DOM — a representação em árvore que o navegador constrói a partir do HTML. HTML semântico bem estruturado torna essa seleção mais precisa e robusta.

8. Boas Práticas: Checklist

Ao concluir qualquer página HTML, percorra este checklist antes de publicar ou entregar. Cada item representa uma prática que, se ignorada, pode resultar em problemas reais: páginas quebradas em celulares, textos ilegíveis para usuários com deficiência, penalidades em mecanismos de busca ou vulnerabilidades de segurança.

Prática
<!DOCTYPE html> na primeira linha. <html lang="pt-BR" com o idioma correto. <meta charset="UTF-8" como primeiro elemento do <head>. <meta name="viewport" content="width=device-width, initial-scale=1.0"». <title> único e descritivo, com menos de 60 caracteres por página. <meta name="description" com resumo de 50160 caracteres. Exatamente um <h1> por página, descrevendo o conteúdo principal. Hierarquia de títulos (h1 h2 h3) sem pular níveis. Todas as com alt descritivo ou alt= para decorativas. Todos os <input> com <label> associado via for/id. Links com texto descritivo (sem “clique aqui” ou “saiba mais”). Links externos com rel="noopener noreferrer" quando target="_blank". Estrutura semântica: <header>, <nav>, <main>, <footer>. CSS em arquivo externo (.css), não inline. JavaScript em arquivo externo (.js) com defer ou no final do <body>. Indentação consistente (2 ou 4 espaços). Validação pelo W3C Validator (validator.w3.org).

Tabela 5. Checklist de boas práticas para um documento HTML bem escrito.

 **Use o W3C Validator como parte do fluxo de trabalho.** O validador do W3C (<https://validator.w3.org>) analisa o HTML de uma URL pública ou de um arquivo enviado e lista todos os erros e avisos de acordo com a especificação oficial. Muitos erros que os navegadores toleram silenciosamente — tags mal aninhadas, atributos inválidos, elementos obsoletos — são detectados pelo validador. Execute-o ao menos uma vez por página antes de considerar o trabalho concluído.

9. Servindo HTML pelo Back-end com Express

Nos handouts anteriores, trabalhamos com o Express exclusivamente como servidor de uma API REST que retorna JSON. No entanto, o Express também pode **servir arquivos HTML** diretamente ao navegador — seja como arquivos estáticos, seja gerados dinamicamente a partir de dados do banco. Essa abordagem é chamada de *server-side rendering* (SSR) e é a forma mais simples de unir back-end e front-end sem precisar de uma SPA (*Single Page Application*).

SSR vs. SPA: quando usar cada abordagem No *server-side rendering* (SSR), o servidor gera o HTML completo e o envia ao navegador — mais simples e melhor para SEO. Em uma *Single Page Application* (SPA), o servidor envia uma página quase vazia e um bundle JavaScript; o navegador constrói o HTML dinamicamente. SPAs (React, Vue, Angular) oferecem mais interatividade, mas exigem mais complexidade. Para aplicações com formulários, listagens e conteúdo público, como um sistema acadêmico simples, SSR com Express e EJS é uma escolha sólida e muito mais rápida de implementar.

9.1. Servindo arquivos estáticos com `express.static`

A forma mais direta de servir HTML, CSS, JavaScript e imagens é declarar uma pasta pública com o middleware `express.static`. Tudo que estiver nessa pasta fica acessível diretamente pelo navegador, sem precisar de nenhuma rota explícita:

```
projeto/  
+-- src/  
|   +-- app.js  
+-- public/          <- pasta servida pelo express.static  
|   +-- index.html  
|   +-- sobre.html  
|   +-- css/  
|       |   +-- estilo.css  
|   +-- js/  
|       |   +-- script.js  
|   +-- img/  
|       |   +-- logo.png  
+-- package.json
```

Código 23. Estrutura de pastas para arquivos estáticos.

```
const express = require('express');  
const path    = require('path');  
const app     = express();  
  
// Serve tudo dentro de /public como arquivos estáticos  
// GET /index.html    -> public/index.html  
// GET /css/estilo.css -> public/css/estilo.css  
app.use(express.static(path.join(__dirname, '..', 'public')));  
  
app.listen(3000, () => console.log('Servidor em http://localhost:3000'));
```

Código 24. Registrando a pasta public no app.js.

💡 **path.join garante caminhos corretos em qualquer sistema operacional.** `path.join(__dirname, '..', 'public')` constrói o caminho absoluto até a pasta `public` de forma segura, independentemente de o servidor rodar no Windows (barras invertidas) ou no Linux/macOS (barras normais). Evite concatenar strings de caminho manualmente (`__dirname + '/../public'`).

9.2. Enviando um arquivo HTML por uma rota com `res.sendFile`

Quando você precisa de controle sobre qual arquivo é servido para qual rota — por exemplo, para proteger certas páginas com autenticação — use `res.sendFile()` dentro de uma rota Express:

```
const express = require('express');
const path = require('path');
const router = express.Router();
const autenticar = require('../middlewares/autenticar');

// Rota pública: entrega o HTML de login
router.get('/login', (req, res) => {
  res.sendFile(path.join(__dirname, '..', '..', 'public', 'login.html'));
});

// Rota protegida: só entrega o HTML do dashboard se autenticado
router.get('/dashboard', autenticar, (req, res) => {
  res.sendFile(path.join(__dirname, '..', '..', 'public', 'dashboard.html'));
});

module.exports = router;
```

Código 25. Rotas que entregam arquivos HTML com `res.sendFile`.

res.sendFile vs. express.static `express.static` serve todos os arquivos da pasta automaticamente, sem passar por nenhum middleware de autenticação. Qualquer pessoa que souber o nome do arquivo pode acessá-lo diretamente. Para páginas que exigem autenticação, use `res.sendFile()` dentro de uma rota protegida pelo middleware `autenticar`. Páginas públicas (login, cadastro, landing page) podem ficar na pasta estática.

9.3. Gerando HTML dinamicamente com template engines

Arquivos HTML estáticos não podem exibir dados do banco de dados. Para gerar HTML com conteúdo dinâmico — como uma lista de alunos vinda do banco — usa-se uma **template engine**: uma biblioteca que combina um template HTML com dados JavaScript para produzir o HTML final no servidor.

O EJS (*Embedded JavaScript*) é a template engine mais simples do ecossistema Node.js. Sua sintaxe usa `<% %>` para código JavaScript e `<%= %>` para exibir valores:

```
npm install ejs    # com npm
yarn add ejs      # com Yarn
```

Código 26. Instalação do EJS.

```
const express = require('express');
const path = require('path');
const app = express();

// Define o EJS como engine de templates
app.set('view engine', 'ejs');

// Define a pasta onde ficam os templates (padrão: ./views)
app.set('views', path.join(__dirname, '..', 'views'));

app.use(express.json());
module.exports = app;
```

Código 27. Configurando o EJS como view engine no `app.js`.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Lista de Alunos</title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <h1>Alunos Matriculados</h1>
  <ul>
    <% alunos.forEach(function(aluno) { %>
      <li><%= aluno.nome %> -- <%= aluno.curso %></li>
    <% }); %>
  </ul>
</body>
</html>
```

Código 28. `views/alunos.ejs` — template com dados dinâmicos.

```
// src/controllers/alunoViewController.js
const db = require('../models');

async function listarAlunos(req, res) {
  // Busca os dados no banco normalmente
  const alunos = await db.Aluno.findAll({ order: [['nome', 'ASC']] });

  // res.render() combina o template com os dados e envia o HTML resultante
  // Primeiro argumento: nome do arquivo em views/ (sem a extensão .ejs)
  // Segundo argumento: objeto com os dados disponíveis no template
  res.render('alunos', { alunos });
}

module.exports = { listarAlunos };
```

Código 29. Controller que busca dados e renderiza o template.

Abordagem	Como funciona	Quando usar	Limitação
express.static	Serve arquivos diretamente da pasta public.	Páginas públicas, CSS, JS, imagens.	Sem controle de acesso por rota.
res.sendFile()	Rota entrega um arquivo HTML fixo.	Páginas com autenticação, sem dados dinâmicos.	HTML não pode incluir dados do banco.
res.render() + EJS	Template HTML + dados JavaScript = HTML final.	Páginas com dados do banco (listas, perfis).	Requer template engine configurada.

Tabela 6. Comparação entre as três abordagens para servir HTML com Express.

⚠ Nunca construa HTML concatenando strings com dados do usuário. Gerar HTML manualmente concatenando dados do usuário diretamente na resposta — como `res.send('<p>' + req.body.nome)` — é vulnerável a ataques XSS (*Cross-Site Scripting*): um usuário malicioso pode injetar código JavaScript nos dados e executá-lo no navegador de outras pessoas. Template engines como o EJS escapam automaticamente os valores inseridos com `%=`, substituindo caracteres perigosos (`<`, `>`, `&`) pelas entidades HTML correspondentes. Use sempre uma template engine ou uma biblioteca de sanitização.

10. Para Pensar e Implementar

Exercício 1 — Estrutura básica: Crie um arquivo `index.html` com a estrutura mínima válida: DOCTYPE, html com lang, head completo (charset, viewport, title, description) e body vazio. Abra no navegador e verifique que a aba exibe o título. Valide o documento em validator.w3.org.

Exercício 2 — Estrutura semântica: Expanda o arquivo anterior adicionando a estrutura semântica completa: `<header>` com logo e título, `<nav>` com três links, `<main>` com um `<section>` contendo dois `<article>`, um `<aside>` com informações complementares e `<footer>` com dados de contato. Não adicione CSS ainda — observe como o navegador renderiza a estrutura pura.

Exercício 3 — Títulos e imagens: Dentro dos artigos criados no exercício anterior, adicione uma hierarquia correta de títulos. Adicione pelo menos duas imagens: uma informativa com alt descritivo e uma decorativa com `alt=`. Teste com um leitor de tela (NVDA no Windows, VoiceOver no macOS) e ouça como o conteúdo é anunciado.

Exercício 4 — Formulário acessível: Crie uma página de login com campos para e-mail e senha. Garanta que cada `<input>` tem um `<label>` associado, que os tipos estão corretos (`type="email"` e `type="password"`) e que o formulário tem botão de envio. Teste a navegação apenas com o teclado (Tab para mover entre campos, Enter para enviar).

Exercício 5 — Auditoria de acessibilidade: Escolha qualquer página web pública. Abra o DevTools do Chrome (F12) e execute uma auditoria com a aba **Lighthouse** (selecione “Accessibility”). Identifique pelo menos três problemas apontados, descreva o que está errado em cada um e proponha a correção em HTML.

Exercício 6 — SSR com Express e EJS: Crie um projeto Express com a estrutura de pastas apresentada neste handout. Implemente uma rota GET `/alunos` que busca registros de um array JavaScript (simulando um banco) e os renderiza com EJS em uma página HTML semântica. Adicione um arquivo `estilo.css` na pasta `public` e referencie-o no template EJS. Garanta que todos os itens do checklist da Tabela 5 estejam atendidos na página gerada.

Resumo dos Pontos-Chave

- **HTML não é linguagem de programação:** é uma linguagem de marcação que descreve a estrutura e o significado do conteúdo. As três camadas da web — HTML, CSS e JavaScript — devem ser mantidas em arquivos separados.
 - **Estrutura obrigatória:** `<!DOCTYPE html>`, `<html lang>`, `<head>` com charset e viewport, `<body>` com conteúdo. A ausência do DOCTYPE ativa o modo quirks.
 - **Head:** contém metadados críticos para SEO e funcionamento correto: `<meta charset>` sempre primeiro, `<meta viewport>` para responsividade, `<title>` único por página e `<meta description>`.
 - **Elementos semânticos:** prefira `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` e `<footer>` a `<div>` genéricas. Use `<div>` e `<span>` apenas quando nenhum elemento semântico for adequado.
 - **Títulos:** um único `<h1>` por página; hierarquia linear sem pular níveis; nunca use headings apenas para efeito visual — isso é papel do CSS.
 - **Imagens:** `alt` descritivo para imagens informativas; `alt=` para decorativas; informe `width` e `height` para evitar layout shift durante o carregamento.
 - **Links:** texto descritivo; `rel="noopener noreferrer"` obrigatório em links com `target="_blank"` para prevenir tabnapping.
 - **Formulários:** sempre associe `<label>` ao `<input>`; use o `type` mais específico; nunca substitua o label pelo placeholder.
 - **Express + HTML:** `express.static` para arquivos públicos; `res.sendFile()` para rotas protegidas; `res.render()` + EJS para conteúdo dinâmico vindo do banco.
 - **Segurança XSS:** nunca concatene dados do usuário diretamente no HTML; use template engines como o EJS, que escapam os valores automaticamente com `<%= %>`.
 - **Validação:** use `validator.w3.org` para validação HTML e o Lighthouse do DevTools para auditorias de acessibilidade, SEO e performance.
-

Referências e Leituras Complementares

- **MDN Web Docs — HTML Reference:** <https://developer.mozilla.org/pt-BR/docs/Web/HTML>
- **MDN — Aprendendo HTML:** <https://developer.mozilla.org/pt-BR/docs/Learn/HTML>
- **W3C Validator:** <https://validator.w3.org> — validação oficial de documentos HTML
- **WHATWG HTML Living Standard:** <https://html.spec.whatwg.org/> — especificação oficial e viva do HTML5
- **WebAIM — Acessibilidade web:** <https://webaim.org/> — guias práticos de acessibilidade
- **Google Lighthouse:** <https://developer.chrome.com/docs/lighthouse> — auditoria de performance, SEO e acessibilidade integrada ao DevTools
- **EJS — Embedded JavaScript Templates:** <https://ejs.co/> — documentação oficial da template engine
- **Express — Serving static files:** <https://expressjs.com/en/starter/static-files.html>
- **OWASP — XSS Prevention Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- **web.dev — Core Web Vitals:** <https://web.dev/vitals/> — métricas de performance do Google, incluindo CLS